

Graph Engineering

Poora Course — Complete Class Notes

(Roman Urdu + English — self-study aur students ko sikhane ke liye)

Ye notes *The AI Agent Factory* ki 'Graph Engineering: A Crash Course' (16 Concepts) par mabni hain — Parts 1 se 7, dogfooding, aur 8 practice projects samet, koi concept chhoote bina. Ye chapter **Loop Engineering** aur **Harness Engineering** ke oopar banti hai — pehle wo dono zaroor dohra lein.

■ Is poore course ka ek jumla: '**Agent bhool jata hai, graph nahi bhoolta.**'

Is Guide mein kya kya hai

- Part 1 — Memory ka masla: transcripts kyun fail hote hain, graph kya hai, do graphs kyun chahiye
- Part 2 — Kaam ka DAG: Karpathy ka autoresearch aur AgentHub
- Part 3 — Facts ka Graph: Anthropic ki extraction, resolution, provenance
- Part 4 — Graph se kaam lena: Subgraphs aur Grounded Checker
- Part 5 — Loops ka Graph: Governance layer, 4 failures, anchors
- Part 6 — Ek Complete Graph: Morning-triage loop ko upgrade karna
- Part 7 — Grounded Rehna: Level chunna, kab graph na banayen
- Dogfooding aur 8 Practice Projects
- Exam-Prep Cheat Sheet

0. Zaroori Terms — Glossary

Term	Aasan matlab
Graph	Points (nodes) jo arrows (edges) se jude hon. Arrows ki direction ka matlab hota hai.
Node	Graph ka ek point — entity, claim, commit, source, agent run.
Edge	Do nodes ke darmiyan ek labeled arrow — jaise 'supports', 'parent_of', 'produced'.
DAG	Directed Acyclic Graph — arrows kabhi wapas apni taraf nahi aate. Git history ek DAG hai.
Commit DAG	Kaam ka graph — commits nodes hain, parent links edges hain. 'Kya try hua?' ka jawab deta hai.
Knowledge Graph	Facts ka graph — entities nodes hain, typed relations edges hain. 'Kya sach hai?' ka jawab deta hai.
Entity	Koi cheez jo graph track karta hai — insaan, company, file, vendor.
Relation/Triple	Ek fact: subject-predicate-object — jaise (Vendor X, supplied, Component Z).
Surface Form	Kisi naam ka jaisa wo document mein likha ho — 'Edwin Aldrin', 'Buzz' — 3 forms, 1 insaan.
Entity Resolution	Faisla karna kaunse surface forms ek hi cheez hain, aur unhe ek canonical node mein jorna.
Provenance	Claim ki 'receipt' — kaunsa source, kaunsa run, kitna confidence.
Claim	Wo statement jo graph mein 'shayad sach' ki tarah store hoti hai — hamesha provenance ke saath.
Subgraph	Graph ka chhota, relevant hissa — ek task ke liye. Poora graph kabhi nahi diya jata.
Grounding	Jawab ko graph ke asal edges se jorhna — apni khud ki impression se nahi.
Swarm	Bohat saare agents jo saath explore/implement/evaluate kar rahe hon.
Structured Output	Model ka jawab ek schema tak mehdood — code isay validate kar sake.
MCP	Model Context Protocol — agent bahar ke system tak pohanchne ka standard tareeqa.
Governance Graph	Wo graph jiske nodes khud loops hain — kaun kise check karta hai, kaun kise feed karta hai.
Execution Loop	Loop jo baar baar kaam karta hai — jaise issues triage karna.
Improvement Loop	Loop jo ek number ko target ke against dekhta hai aur system ko adjust karta hai.
Counter-metric	Doosra number, jo pehle number ko 'game' hone se bachata hai.
Anchor	Wo measurement jis se koi loop bahas nahi kar sakta — asal test, asal customer, asal paisa.
Frozen Node	Wo rule/file jo optimizing loops kabhi nahi badal sakte.

Part 1 — Memory Ka Masla

1. Agent Bhool Jata Hai, Graph Nahi Bhoolta

Aapka morning-triage loop ek spine rakhta hai (progress.md) — ek loop ke liye ye kaafi hai. Lekin socho: aap ek doosra loop add karte hain, phir ek review loop, phir 20 agents ek saath 20 files audit karne ke liye chhod dete hain. Har agent khali context se shuru hota hai. Agent 7 ek bug dhoondta hai. Agent 14, ek ghante baad, wahi bug dobara dhoondta hai. Kaam barh gaya, memory nahi barhi.

progress.md ek loop ke liye theek hai, lekin ye cheezein nahi kar sakta:

- **Doosra loop is par bharosa nahi kar sakta** — ye prose hai, format kabhi bhi badal sakta hai
- **20 parallel agents share nahi kar sakte** — har koi khali shuru hota hai
- **Aap query nahi kar sakte** — 'Pichle mahine ke findings jo payment code se related thay aur reviewer ne confirm kiye?' — spine is ka jawab nahi de sakti
- **Ye kisi cheez ka source nahi deti** — 'flaky test fix kiya' — kaunsa test? kis run ne prove kiya?

■ Hal: agents jo seekhein usay **typed records** ki tarah likhein — entity, claim, source, run — labeled arrows se jude hue. Koi bhi baad wala agent isay query kar sakta hai.

2. Graph Kya Hai — Nodes, Edges, Direction

Teen cheezein poora kaam karti hain:

- **Nodes typed hote hain** — sirf 'box' nahi, balke 'Entity', 'Claim', 'Source', 'Commit'
- **Edges labeled aur directed hote hain** — (claim_441) →supported_by→ (source) — direction hi matlab hai
- **Paths jawab hote hain** — 'Kya Vendor X, Incident Y se juda hai?' ban jata hai 'kya edges ka koi raasta maujood hai?'

■ Prose ko padhna aur samajhna padta hai. Graph ko **traverse** kiya ja sakta hai — mechanically, code se, har baar wahi tareeqa. Jab memory graph ban jati hai, sawal jo 'vibe' thay, ab query ban jate hain.

DAG (Directed Acyclic Graph) ek khaas shape hai — arrows kabhi wapas nahi aate. Git history isi ki misaal hai.

3. Do Graphs — Inhen Mila Mat Do

	Commit DAG (Part 2)	Knowledge Graph (Part 3)
Yaad rakhta hai	Kaam — kya try hua	Facts — kya maloom hai
Nodes	Commits, experiments, runs	Entities, claims, sources
Edges	parent_of, derived_from	supports, works_for, about
Truth ka standard	Fact by construction — hua, Git guarantee karta hai	Statement with evidence — ghalat ho sakta hai

Dono jud sakte hain: (agent_run) →produced→ (claim) →about→ (entity). Ek run ne kaam kiya (commit) aur kuch seekha (claim). Kaam ki lineage ek taraf, domain knowledge doosri taraf — jude hue, lekin mila hue nahi.

■ Agar aapke tasks independent hain, jawab ek document se aate hain, relations simple hain, aur kisi ko provenance nahi chahiye — **rukiye yahin**. Ek spine wala loop hi kaafi hai. Part 7 isay 6 sawalon mein tabdeel karta hai.

Part 2 — Kaam Ka DAG

4. Autoresearch — Ratchet Apni History Git Mein Likhta Hai

Karpathy ka **autoresearch** (7 March 2026) wahi ratchet loop hai jo aap jaante hain, ML training par lagaya gaya. Iski khaas baat: **iski memory transcript nahi, commit DAG hai**.

Teen files: `prepare.py` (frozen — agent chhoo nahi sakta), `train.py` (jahan agent kaam karta hai), `program.md` (instructions). Har beat: ek change try karo, commit karo, train karo, metric check karo. Behtar? Rakho. Bura/crash? Reset. Ya to wapas, ya aage.

	Kya rakhta hai	Truth standard
Git branch	Sirf retained improvements, chain mein	Verified — har commit ne metric behtar kiya
results.tsv	HAR attempt — kept, discarded, crashed	Complete — attempts ka record, achievements ka nahi

■ Jab reset hota hai, wo commit sirf branch se hat jata hai — TSV mein reh jata hai. Discarded idea sirf ek text row banti hai jise koi doosra agent query nahi kar sakta. Yehi gap Concept 5 bharta hai.

5. AgentHub — Search Graph Traverse Karo, Main Mein Merge Mat Karo

Karpathy ka agla idea: loop ko ek **community** banao. AgentHub: ek Go binary, SQLite, bare Git repo, message board. Slogan: **'GitHub insaano ke liye hai. AgentHub agents ke liye hai.'**

Asal move: reset karna band karo, **rakhna** shuru karo — har commit ek durable node banta hai, kisi main branch par converge hone ki zaroorat nahi.

```
ah push # apna commit ek naye node ki tarah publish karo
ah children # is result ke oopar kya try hua?
ah leaves # frontier — jo abhi tak kisi ne explore nahi kiya
ah lineage # poori ancestry jisne ye outcome banaya
```

■ Ghalat experiment bhi ek node banta hai — 'batch 64 memory limit se tajawuz karta hai' — is se agla agent bina crash dohraye seekh leta hai. Yehi swarm aur crowd mein farq hai.

Nota bene: AgentHub abhi private hai — 404 deta hai. Sirf design idea seekhein, ise dependency ki tarah use na karein.

Part 3 — Facts Ka Graph

Git ne commit DAG muft di. Knowledge graph aapko khud banana padta hai. Anthropic ki **Knowledge Graph Construction Cookbook** (23 March 2026) is kaam ko 4 stages mein todhti hai — koi trained NLP model nahi chahiye, sirf structured-output prompts.

6. Extraction — Schema hi 'Training Data' Hai

Ek Pydantic schema define karo (Entity, Relation), aur model ka jawab usi tak mehdood karo.

- **Schema hi training data hai** — koi labeled corpus nahi chahiye
- **Sasta model volume kaam karta hai** — extraction Haiku par chalta hai, mehnga model resolution (agla concept) mein
- **Description field decoration nahi** — ye entity ke around context deti hai, jo resolution ke liye zaroori hai

7. Resolution — Ek Cheez, Kai Naam

Extraction 'surface forms' deta hai — 'Edwin Aldrin', 'Buzz Aldrin', 'Col. Aldrin' — teen naam, ek insaan. String similarity kaam nahi karti: alag naam ka match miss ho jata hai, do alag Muhammad Khan match ho jate hain.

■ Hal: resolution ko **reasoning task** ki tarah treat karo. Ek strong model ko description ke saath entities dikhao, canonical clusters banwao. Usool: **resolution additive aur reversible honi chahiye** — aliases kabhi mitao mat, sirf link karo. False merge sabse bara khatra hai.

8. Provenance — Har Edge Apni Receipt Rakhti Hai

Assembly stage mechanically simple hai, lekin har node/edge ko ye zaroor carry karna chahiye — 4 invariants:

- Har **claim** ka ek source ho, ya 'inference' mark ho
- Har **artifact** ka ek authoring run aur version ho
- Har **evaluation** apna rubric bataye
- Har **superseded** object addressable rahe — replace ho, mitaya nahi jaye

■ Graph **truth machine nahi hai** — ye claims, sources, aur relationships ko inspectable rakhta hai. Iska honest wada chhota hai: **kuch bhi bina attribution ke nahi, kuch bhi bina jaanche ke nahi.**

Part 4 — Graph Se Kaam Lena

9. Subgraph, Poora Graph Nahi

Graph banane ka maqsad context-dumping se bachna tha. Isay barbaad karne ka sabse tez tareeqa: poora graph har prompt mein daal dena. Discipline hai **bounded retrieval** — har worker ko task-specific subgraph milta hai, aur bas.

- **Resolve** — task jo entities mention karta hai unhe graph mein dhoondo
- **Expand** — 1-2 hops, sirf allowed edge types par
- **Include** — current artifact versions
- **Prioritize** — recent, verified claims pehle
- **Conflicts shamil karo** — agar do claims takrayen, worker ko dono dikhao
- **Token budget mein serialize karo** — plain triples ki tarah
- **Stable edge IDs attach karo** — taake checker cite kar sake

■ Naye employee ko poori filing cabinet mat do — sirf 3 folders jo unke task se related hain, plus sticky note 'ye do folders aapas mein takrate hain: check karo.'

10. Grounded Checker — 'Triple Not Found' > 'Seems Off'

Loop course ka maker-checker, harness course ka typed output — lekin checker ka verdict phir bhi ek *impression* tha. Graph ye badalta hai: checker **edges ke against** check kar sakta hai.

Maker kehta hai: 'Vendor X ne wo component supply kiya jo Incident Y mein tha.' Grounded checker ye nahi poochta 'sahi lagta hai?' — ye poochta hai: (vendor_x, supplied, component_z) edge maujood hai? Agar nahi, to jawab hota hai:

```
{ "decision": "revise", "reason": "vendor_x se incident_y tak koi supported path nahi",  
  "required_evidence": ["supplied relation", "involved_in relation"] }
```

■ 'Seems off' maker ko guess karne bhejta hai. 'Triple not found: ye 2 edges chahiye' ek work order hai. Ek mood hai, doosra kaam hai. Aur jab maker evidence dhoond leta hai, graph ko ek nayi edge milti hai — grounding memory ko behtar banati hai, sirf report ko nahi.

■ Exam tip — Sawal: 'Grounded checker sab claims par PASS deta hai, ek hafte baad ek claim ghalat nikalti hai — do jagah galti ho sakti hai?' Jawab: **ya to graph ghalat tha** (bad extraction/false merge — Part 3 fix), **ya checker ghalat tha** (galat edge cite ki — ye 'Trusting the Checker' course ka masla hai).

Part 5 — Loops Ka Graph (Governance)

11. Wiring — Kaun Kise Feed Karta Hai, Kaun Kise Check Karta Hai

Governance graph mein **loop khud ek node hai**, aur graph unke darmiyan wiring hai. Maker-checker split ek edge hai. Human gate ek node hai. Dreaming loop bhi ek graph hai.

Loop Type	Kya karta hai
Execution loop	Baar baar kaam karta hai — issues triage, PR review
Improvement loop	Ek number ko target ke against dekhta hai, system adjust karta hai

Teen sawal har edge par poochein: **Routing** (result kise jaye?), **Trust boundaries** (kaunsa loop kise fire kar sakta hai?), **Gate placement** (insaan kahan khada ho?).

■ Model ko **classify** karne dein (probabilistic), lekin **route table** deterministic honi chahiye — low risk seedha raasta, high risk poora audit. Model ka judgment lein, uski improvisation nahi.

12. Perez Ke 4 Failures (Single Loop Ke)

Failure	Kya hota hai	Graph ka jawab
Gaming	Loop apna number ghalat tareeqe se badhata hai (Goodhart's law)	Har optimizing loop ke saath ek watching loop (counter-metric)
Blindness upward	Loop apne hi target par sawal nahi utha sakta	Dheema loop tez loop ke target ka malik ho
Conflict	Alag banaye loops aapas mein larte hain	Ek arbitration node (insaan/supervisor)
Measurement decay	Check karna 'reality check' se 'report vs report' ban jata hai	Independent audit loops

■ Support-bot ki kahani: 'tickets resolved' number barhta gaya, lekin bot customers ko bhaga kar tickets band kar raha tha. Fix kabhi 'behtar loop' nahi hota — hamesha ek **edge** hota hai.

13. Anchors aur Frozen Nodes — Graph Khud Ko Bhi Dhoka De Sakta Hai

Agar har loop sirf doosre loop ki report padhta hai — sab aapas mein 'agree' karte hain, lekin kisi ko reality se check nahi kiya gaya. Isay **circular graph** kehte hain — ye single-loop wale masle ki tarah fail hota hai, sirf baad mein aur mehenga.

- **Anchors** — jo measurement koi loop challenge nahi kar sakta: asal test, asal customer, asal paisa
- **Frozen nodes** — rules jo optimizing loops kabhi badal nahi sakte (check.py, prepare.py)
- **Root judgment bahar se** — 'behtar' ka matlab kya hai — ye faisla sirf insaan de sakte hain

■ Asal sawal kabhi 'loops vs graphs' nahi tha. Asal sawal hamesha ye tha: **grounded ya ungrounded?** Kya aapki machinery reality ko chhooti hai jise wo behtar banane ka daawa karti hai?

Part 6 — Ek Complete Graph

Morning-triage loop ko prose spine se ek chhoti, queryable graph mein upgrade karte hain — sirf 3 JSON files, koi database nahi.

```
graph/
SCHEMA.md # contract: fields, types, write rules
entities.json # nodes
claims.json # edges-with-receipts
runs.json # kaunsa beat ne kya likha
evidence/
run_log.txt # raw tool output jise claim point kar sake
```

Ek claim ki misaal:

```
{ "id": "claim_0007", "subject": "test_payments_flaky",
  "predicate": "diagnosed_as", "object": "tz_default_utc",
  "source": { "kind": "tool_output", "command": "pytest ...",
    "exit_code": 1, "ref": "evidence/run_log.txt#L88-94"},
  "produced_by": "run_2026-07-21-triage", "supersedes": "claim_0004" }
```

Do zaroori usool

- **Koi 'status' field nahi** — claims append-only hain. 'Active' hone ka faisla read-time par derive hota hai: jise koi doosri claim supersede na kare, wo active hai
- **Predicate 'diagnosed_as' hai, 'caused_by' nahi** — evidence jo prove kar sakti hai, wahi likhein
- **Source tool ka naam le, agent ka nahi** — 'tool_output' wo hai jo model ne nahi likha (pytest, API, OS). Agent ka apna prose evidence nahi ban sakta

■ **Maker** (kaam karne wala) claims.json mein likhta hai — append-only, source ke saath, ID counter se nahi balke derived (taake retry duplicate na bana de). Reviewer graph se padhta hai — har factual claim ko cite karna zaroori, warna REVISE.

Pehle (sirf spine): Mangal ko fix hua, prose likha gaya. Teen hafte baad koi poochta hai 'kaunsi timezone thi?' — transcripts mein khudai karni padti hai.

Baad mein (graph): Mangal ko claim_0007 likha jata hai, source ke saath. Teen hafte baad, jawab ek jq command se milta hai — ek line, receipt ke saath.

Part 7 — Grounded Rehna

14. Level Chunna — 6 Sawal Pehle Poochein

- Success verify ho sakta hai?
- Steps stable hain?
- Subtasks independent hain?
- Alternative lineages zinda rehni chahiye?
- Facts run ke baad bhi zinda rehne chahiye?
- Cost aur latency afford kar sakte hain?

Situation	Shuru karein	Kyun
Simple sawal	Zero-shot	Sabse kam latency
Output check ho sakta hai	Loop	Feedback se behtar hota hai
Sequence stable hai	Chain	Predictable stages
Alternatives zinda rehna zaroori	Commit DAG	Experiment branches mehfooz
Facts session se aage rehne chahiye	Knowledge Graph	Persistent shared memory

■ Graph 8th row hai, pehla nahi. Zyada tar kaam pehle hi ruk jata hai — **ye ambition ki kami nahi, sahi faisla hai**. Har run ke liye pehle se **complexity budget** declare karein: max calls, max workers, max tokens, max cost — aur minimum evidence jo 'done' kehne ke liye chahiye.

Anthropic ke apne multi-agent system ne single agent se behtar kaam kiya — lekin **15 guna zyada tokens** use kiye. Isliye: sasta model chhote kaam ke liye, strong model synthesis/decomposition ke liye, aur poora graph sirf tab jab kaam ka faida uski coordination cost se zyada ho.

15. Kab Graph Na Banayen

- Tasks independent hain, cross-session state nahi chahiye
- Jawab ek document se aata hai
- Relations fixed aur simple hain — relational table kaafi hai
- Provenance ki zaroorat nahi
- Extraction errors traversal ke faide se zyada honge

■ Graph builder ki judgment amplify karta hai — **ghalat judgment bhi**. Ghalat entity types, ghalat sources — automation galti ko scale kar deta hai. Har optimization ka apna counter-metric chahiye (precision vs recall).

16. Graph Kya Nahi Kar Sakta

- **'Checker ka PASS ab bharosemand hai'** — Nahi. Checker phir bhi ek model hai, ghalat edge cite kar sakta hai
- **'Memory apni jagah mehfooz hai'** — Sirf jitni uski home mehfooz hai. Laptop ka repo laptop ke saath mar jata hai
- **'Behtar wiring ka matlab behtar judgment'** — Ontology, source policy, anchors, aur 'behtar' ka matlab — sab bahar se, insaan se aata hai

■ Asal bottleneck aksar agli model call nahi hai — **memory aur evaluation ki placement hai**. Lekin 'behtar' ka matlab koi graph nahi de sakta — wo sirf aap de sakte hain.

Practice Projects

#	Naam	Kya Prove Karta Hai
1	Draw your system	Apna poora setup nodes/edges ki tarah kheenchna
2	Spine to claims	10 findings ko typed claims mein badalna
3	First extraction	Schema-constrained prompt se entities nikalna
4	The DAG speaks	AgentHub ke 3 sawal plain Git se poochna
5	Resolution drill	Sahi merge, ghalat merge se bachna
6	The grounded reviewer	Checker ko edge maangne par majboor karna
7	A gold set for extraction	Extraction pipeline ko measure karna
8	Two loops, one graph (Capstone)	Do loops, ek shared graph, poora traceable

Exam-Prep Cheat Sheet

- **Agent bhool jata hai, graph nahi bhoolta.**
- **Do graphs:** Commit DAG (kaam) aur Knowledge Graph (facts) — mila mat do.
- **Graph ke 3 usool:** Nodes typed hain, edges labeled/directed hain, paths jawab hain.
- **Autoresearch:** Git branch (retained), results.tsv (har attempt).
- **AgentHub:** Reset nahi, keep — har commit ek node banta hai.
- **Extraction:** Schema hi training data. Cheap model volume ke liye.
- **Resolution:** Additive + reversible. False merge sabse bara khatra.
- **4 Provenance Invariants:** source, authoring run, rubric, supersession (kabhi mitao mat).
- **Subgraph:** Poora graph kabhi nahi — resolve, expand, prioritize, conflicts shamil, budget mein serialize.
- **Grounded checker:** 'Triple not found' — mood nahi, work order.
- **4 Failures:** Gaming, Blindness upward, Conflict, Measurement decay — fix hamesha edge hai.
- **anchors + Frozen nodes:** Circular graph se bachne ka ilaj — reality ko chhoona zaroori.
- **6 Sawal pehle poochein** level chunne se pehle — graph 8th row hai, pehla nahi.
- **Complexity budget:** har run se pehle limits declare karein.

Ye notes The AI Agent Factory ki Graph Engineering crash-course par mabni hain, apne alfaz mein students ke liye tayyar kiye gaye — poori tafseel ke liye asal chapter dekhien:
agentfactory.panaversity.org/docs/graph-engineering-crash-course